

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №2

По дисциплине «**Распределённые системы хранения данных**»

Выполнил:

Студент группы Р3306,
Михайлов Дмитрий Андреевич

Преподаватель:

Максимов Андрей Николаевич



Санкт-Петербург

2026 год

Содержание

Задание	3
Подготовка окружения	4
Этап 1. Резервное копирование.....	5
1.1. Проверка основного узла.....	5
1.2. Подготовка каталога на резервном узле	5
1.3. Настройка SSH-доступа для автоматической передачи архивов	5
1.4. Сценарий резервного копирования	6
1.5. Проверка сценария резервного копирования	7
1.6. Настройка периодического запуска через cron	8
1.7. Расчет объема резервных копий за месяц.....	9
1.8. Анализ результатов этапа.....	9
Этап 2. Потеря основного узла	10
2.1. Подготовка резервного узла и выбор резервной копии.....	10
2.2. Инициализация восстановленного кластера	11
2.3. Конфигурация и запуск сервера на резервном узле	12
2.4. Восстановление из SQL Dump-архива	13
2.5. Проверка доступности данных	14
2.6. Анализ результатов этапа.....	16

Задание

Цель работы - настроить процедуру периодического резервного копирования базы данных, сконфигурированной в ходе выполнения лабораторной работы №2, а также разработать и отладить сценарии восстановления в случае сбоя.

Узел из предыдущей лабораторной работы используется в качестве основного. Новый узел используется в качестве резервного. Учётные данные для подключения к новому узлу выдаёт преподаватель. В сценариях восстановления необходимо использовать копию данных, полученную на первом этапе данной лабораторной работы.

Требования к отчёту

Отчет должен быть самостоятельным документом (без ссылок на внешние ресурсы), содержать всю последовательность команд и исходный код скриптов по каждому пункту задания. Для демонстрации результатов приводить команду вместе с выводом (самой наглядной частью вывода, при необходимости).

Этап 1. Резервное копирование

- Настроить резервное копирование с основного узла на резервный следующим образом:

Периодические полные копии с помощью SQL Dump.

По расписанию (cron) раз в сутки, методом SQL Dump с сжатием. Созданные архивы должны сразу перемещаться на резервный хост, они не должны храниться на основной системе. Срок хранения архивов на резервной системе - 4 недели. По истечении срока хранения, старые архивы должны автоматически уничтожаться.

- Подсчитать, каков будет объем резервных копий спустя месяц работы системы, исходя из следующих условий:
 - Средний объем новых данных в БД за сутки: 650МБ.
 - Средний объем измененных данных за сутки: 750МБ.
- Проанализировать результаты.

Этап 2. Потеря основного узла

Этот сценарий подразумевает полную недоступность основного узла. Необходимо восстановить работу СУБД на РЕЗЕРВНОМ узле, продемонстрировать успешный запуск СУБД и доступность данных.

Этап 3. Повреждение файлов БД

Этот сценарий подразумевает потерю данных (например, в результате сбоя диска или файловой системы) при сохранении доступности основного узла. Необходимо выполнить полное восстановление данных из резервной копии и перезапустить СУБД на ОСНОВНОМ узле.

Ход работы:

- Симулировать сбой:
 - удалить с диска директорию любого табличного пространства со всем содержимым.
- Проверить работу СУБД, доступность данных, перезапустить СУБД, проанализировать результаты.

- Выполнить восстановление данных из резервной копии, учитывая следующее условие:
 - исходное расположение директории PGDATA недоступно - разместить данные в другой директории и скорректировать конфигурацию.
- Запустить СУБД, проверить работу и доступность данных, проанализировать результаты.

Этап 4. Логическое повреждение данных

Этот сценарий подразумевает частичную потерю данных (в результате нежелательной или ошибочной операции) при сохранении доступности основного узла. Необходимо выполнить восстановление данных на ОСНОВНОМ узле следующим способом:

- Восстановление с использованием архивных WAL файлов. (СУБД должна работать в режиме архивирования WAL, потребуется задать параметры восстановления).

Ход работы:

- В каждую таблицу базы добавить 2-3 новые строки, зафиксировать результат.
- Зафиксировать время и симулировать ошибку:
 - удалить любые две таблицы (DROP TABLE)
- Продемонстрировать результат.
- Выполнить восстановление данных указанным способом.
- Продемонстрировать и проанализировать результат.

Подготовка окружения

```
mysticslippers@LAPTOP-GD639C6U:~$ ssh -J s368530@helios.cs.ifmo.ru:2222
postgres3@pg111

[postgres3@pg111 ~]$ export PGDATA="$HOME/ymy46"      #Директория кластера
[postgres3@pg111 ~]$ export PGPORT="9530"           #Номер порта
[postgres3@pg111 ~]$ export PGDATABASE="fastorangecity"
```

Этап 1. Резервное копирование

Основной узел: postgres3@pg111. Резервный узел: postgres4@pg121. Кластер PostgreSQL на основном узле был создан в предыдущей лабораторной работе в каталоге \$HOME/ymy46 и работает на порту 9530. Целевая база данных: fastorangecity.

1.1. Проверка основного узла

Сначала на основном узле были заданы переменные окружения для работы с экземпляром PostgreSQL:

```
[postgres3@pg111 ~]$ export PGDATA="$HOME/ymy46"
[postgres3@pg111 ~]$ export PGPORT="9530"
[postgres3@pg111 ~]$ export PGDATABASE="fastorangecity"
```

После этого был выполнен запуск сервера PostgreSQL:

```
[postgres3@pg111 ~]$ pg_ctl -D "$PGDATA" start
ожидание запуска сервера...2026-04-26 10:27:23.387 MSK [61090] СООБЩЕНИЕ: передача
вывода в протокол процессу сбора протоколов
2026-04-26 10:27:23.387 MSK [61090] ПОДСКАЗКА: В дальнейшем протоколы будут
выводиться в каталог "log".
    готово
сервер запущен
```

Доступность базы данных была проверена с помощью psql:

```
[postgres3@pg111 ~]$ psql -p "$PGPORT" -d fastorangecity -c '\conninfo'
Вы подключены к базе данных "fastorangecity" как пользователь "postgres3" через сокет
в "/tmp", порт "9530".
```

Таким образом, основной экземпляр PostgreSQL был запущен и база данных fastorangecity доступна для подключения.

1.2. Подготовка каталога на резервном узле

На резервном узле был создан отдельный каталог для хранения сжатых SQL Dump-архивов:

```
mysticslippers@LAPTOP-GD639C6U:~$ ssh -J s368530@helios.cs.ifmo.ru:2222
postgres4@pg121

[postgres4@pg121 ~]$ mkdir -p "$HOME/pg_backups/sql_dump"
[postgres4@pg121 ~]$ chmod 700 "$HOME/pg_backups" "$HOME/pg_backups/sql_dump"
[postgres4@pg121 ~]$ ls -ld "$HOME/pg_backups/sql_dump"
drwx----- 2 postgres4 postgres 2 26 apr. 10:31
/var/db/postgres4/pg_backups/sql_dump
```

1.3. Настройка SSH-доступа для автоматической передачи архивов

Так как резервное копирование должно выполняться автоматически по расписанию, был настроен SSH-доступ по ключу с основного узла на резервный. Это необходимо, так как cron не может интерактивно вводить пароль.

На основном узле был создан ключ для передачи резервных копий:

```
[postgres3@pg111 ~]$ mkdir -p ~/.ssh
[postgres3@pg111 ~]$ chmod 700 ~/.ssh
[postgres3@pg111 ~]$ ssh-keygen -t ed25519 -f ~/.ssh/backup_pg121_key -N ""
Generating public/private ed25519 key pair.
Your identification has been saved in /var/db/postgres3/.ssh/backup_pg121_key
Your public key has been saved in /var/db/postgres3/.ssh/backup_pg121_key.pub
```

```
The key fingerprint is:  
SHA256:uPxDf1HK4BAWi6H3tLZgWalfcWl12doU9d+R8qEKiOI postgres3@pg111.cs.ifmo.ru
```

Публичный ключ был добавлен в файл ~/.ssh/authorized_keys на резервном узле pg121 и на промежуточном узле helios. Это позволило выполнять подключение по цепочке pg111 -> helios -> pg121 без ввода пароля.

Для удобства был создан SSH-конфигурационный файл на основном узле:

```
[postgres3@pg111 ~]$ cat > ~/.ssh/config <<'EOF'  
Host helios_jump  
  HostName helios.cs.ifmo.ru  
  Port 2222  
  User s368530  
  IdentityFile ~/.ssh/backup_pg121_key  
  IdentitiesOnly yes  
  
Host backup_pg121  
  HostName pg121  
  User postgres4  
  ProxyJump helios_jump  
  IdentityFile ~/.ssh/backup_pg121_key  
  IdentitiesOnly yes  
  ServerAliveInterval 30  
  ServerAliveCountMax 3  
EOF  
  
[postgres3@pg111 ~]$ chmod 600 ~/.ssh/config
```

Была выполнена проверка подключения к промежуточному узлу helios:

```
[postgres3@pg111 ~]$ ssh helios_jump 'hostname; whoami'  
helios.cs.ifmo.ru  
s368530
```

После этого была проверена полная цепочка подключения до резервного узла:

```
[postgres3@pg111 ~]$ ssh backup_pg121 'hostname; pwd; ls -ld ~/pg_backups/sql_dump'  
pg121.cs.ifmo.ru  
/var/db/postgres4  
drwx----- 2 postgres4 postgres 2 26 apr. 10:31  
/var/db/postgres4/pg_backups/sql_dump
```

Команда выполнилась без запроса пароля, следовательно, автоматическая передача архивов с основного узла на резервный возможна из cron.

1.4. Сценарий резервного копирования

На основном узле был создан каталог для сценариев и журналов выполнения:

```
[postgres3@pg111 ~]$ mkdir -p "$HOME/scripts" "$HOME/backup_logs"
```

Далее был создан сценарий backup_sql_dump.sh:

```
[postgres3@pg111 ~]$ cat > "$HOME/scripts/backup_sql_dump.sh" <<'EOF'  
#!/bin/sh  
  
set -eu  
  
export PGDATA="$HOME/ymy46"  
export PGPORT="9530"  
  
BACKUP_HOST="backup_pg121"  
REMOTE_DIR="/var/db/postgres4/pg_backups/sql_dump"
```

```

DATE_TAG="$(date '+%Y-%m-%d_%H-%M-%S')"
BACKUP_NAME="pg111_full_cluster_${DATE_TAG}.sql.gz"
LOG_FILE="$HOME/backup_logs/backup_sql_dump.log"

echo "[$(date '+%Y-%m-%d %H:%M:%S')] backup started: ${BACKUP_NAME}" >> "$LOG_FILE"

pg_ctl -D "$PGDATA" status >> "$LOG_FILE" 2>&1

ssh "$BACKUP_HOST" "mkdir -p '$REMOTE_DIR' && chmod 700 '$REMOTE_DIR'"

pg_dumpall -p "$PGPORT" \
| gzip -9 \
| ssh "$BACKUP_HOST" "cat > '$REMOTE_DIR/${BACKUP_NAME}.tmp' && mv '$REMOTE_DIR/${BACKUP_NAME}.tmp' '$REMOTE_DIR/${BACKUP_NAME}'"

ssh "$BACKUP_HOST" "find '$REMOTE_DIR' -type f -name 'pg111_full_cluster_*.sql.gz' -mtime +28 -delete"

ssh "$BACKUP_HOST" "ls -lh '$REMOTE_DIR/${BACKUP_NAME}'" >> "$LOG_FILE" 2>&1

echo "[$(date '+%Y-%m-%d %H:%M:%S')] backup finished: ${BACKUP_NAME}" >> "$LOG_FILE"
EOF

[postgres3@pg111 ~]$ chmod +x "$HOME/scripts/backup_sql_dump.sh"

```

Сценарий работает потоковым способом: вывод `pg_dumpall` передается в `gzip`, после чего сразу записывается на резервный узел через SSH. Поэтому архив резервной копии не создается и не хранится на основном узле. Для защиты от частично записанных файлов сначала создается временный файл с расширением `.tmp`, а затем он переименовывается в итоговый архив.

Удаление архивов старше 28 дней выполняется командой `find` на резервном узле:

```
find '$REMOTE_DIR' -type f -name 'pg111_full_cluster_*.sql.gz' -mtime +28 -delete
```

1.5. Проверка сценария резервного копирования

После создания сценария был выполнен тестовый запуск:

```
[postgres3@pg111 ~]$ "$HOME/scripts/backup_sql_dump.sh"
```

Журнал выполнения показал успешное создание и передачу архива на резервный узел:

```

[postgres3@pg111 ~]$ tail -n 20 "$HOME/backup_logs/backup_sql_dump.log"
[2026-04-26 11:22:51] backup started: pg111_full_cluster_2026-04-26_11-22-51.sql.gz
pg_ctl: сервер работает (PID: 61090)
/usr/local/bin/postgres "-D" "/var/db/postgres3/ymy46"
-rw-r--r--  1 postgres4 postgres  2,4K 26 апр.  11:22
/var/db/postgres4/pg_backups/sql_dump/pg111_full_cluster_2026-04-26_11-22-51.sql.gz
[2026-04-26 11:22:55] backup finished: pg111_full_cluster_2026-04-26_11-22-51.sql.gz
[2026-04-26 11:26:12] backup started: pg111_full_cluster_2026-04-26_11-26-12.sql.gz
pg_ctl: сервер работает (PID: 61090)
/usr/local/bin/postgres "-D" "/var/db/postgres3/ymy46"
-rw-r--r--  1 postgres4 postgres  2,4K 26 апр.  11:26
/var/db/postgres4/pg_backups/sql_dump/pg111_full_cluster_2026-04-26_11-26-12.sql.gz
[2026-04-26 11:26:15] backup finished: pg111_full_cluster_2026-04-26_11-26-12.sql.gz

```

Наличие архивов на резервном узле было проверено командой:

```

[postgres3@pg111 ~]$ ssh backup_pg121 'ls -lh /var/db/postgres4/pg_backups/sql_dump'
total 9
-rw-r--r--  1 postgres4 postgres  2,4K 26 апр.  11:22 pg111_full_cluster_2026-04-26_11-22-51.sql.gz

```

```
-rw-r--r-- 1 postgres4 postgres 2,4K 26 апр. 11:26 pg111_full_cluster_2026-04-26_11-26-12.sql.gz
```

Проверка отсутствия архивов на основном узле:

```
[postgres3@pg111 ~]$ find "$HOME" -maxdepth 3 -name 'pg111_full_cluster_*.sql.gz'
```

Команда не вывела результатов. Следовательно, архивы не сохраняются на основной системе и сразу передаются на резервный узел.

Также была проверена целостность последнего gzip-архива:

```
[postgres3@pg111 ~]$ ssh backup_pg121 'LATEST=$(ls -t /var/db/postgres4/pg_backups/sql_dump/pg111_full_cluster_*.sql.gz | head -n 1); echo "$LATEST"; gzip -t "$LATEST" && echo OK' /var/db/postgres4/pg_backups/sql_dump/pg111_full_cluster_2026-04-26_11-26-12.sql.gz OK
```

Проверка содержимого архива показала, что внутри находится SQL Dump всего кластера PostgreSQL:

```
[postgres3@pg111 ~]$ ssh backup_pg121 'LATEST=$(ls -t /var/db/postgres4/pg_backups/sql_dump/pg111_full_cluster_*.sql.gz | head -n 1); gzip -dc "$LATEST" | head -n 20'
--
-- PostgreSQL database cluster dump
--

SET default_transaction_read_only = off;

SET client_encoding = 'WIN1251';
SET standard_conforming_strings = on;

--
-- Roles
--

CREATE ROLE default_user;
ALTER ROLE default_user WITH NOSUPERUSER INHERIT NOCREATEROLE NOCREATEDB LOGIN
NOREPLICATION NOBYPASSRLS PASSWORD 'SCRAM-SHA-256$...';
CREATE ROLE postgres3;
ALTER ROLE postgres3 WITH SUPERUSER INHERIT CREATEROLE CREATEDB LOGIN REPLICATION
BYPASSRLS;
```

В начале дампа присутствует строка SET client_encoding = 'WIN1251', что соответствует кодировке кластера, заданной при выполнении предыдущей лабораторной работы. Также в дампе присутствует секция Roles, что подтверждает использование pg_dumpall и сохранение глобальных объектов кластера.

1.6. Настройка периодического запуска через cron

Для ежедневного автоматического запуска сценария резервного копирования было создано задание cron. Чтобы избежать ошибок при редактировании crontab, запись была добавлена одной командой:

```
[postgres3@pg111 ~]$ ( crontab -l 2>/dev/null; echo '0 3 * * * /var/db/postgres3/scripts/backup_sql_dump.sh >> /var/db/postgres3/backup_logs/cron_backup.log 2>&1' ) | crontab -
```

Проверка установленного расписания:

```
[postgres3@pg111 ~]$ crontab -l
```



```
0 3 * * * /var/db/postgres3/scripts/backup_sql_dump.sh >>  
/var/db/postgres3/backup_logs/cron_backup.log 2>&1
```

Задание cron выполняет сценарий backup_sql_dump.sh ежедневно в 03:00. Стандартный вывод и сообщения об ошибках перенаправляются в файл /var/db/postgres3/backup_logs/cron_backup.log.

1.7. Расчет объема резервных копий за месяц

По условию задания средний объем новых данных в базе данных за сутки составляет 650 МБ, а средний объем измененных данных за сутки - 750 МБ. При этом используется полное логическое резервное копирование SQL Dump. Это означает, что каждый ежедневный архив содержит полное состояние кластера на момент копирования, а не только изменения за сутки.

Пусть S_0 - начальный размер логического дампа. Тогда размер полного дампа в n -й день можно оценить как:

$$S(n) = S_0 + 650 * n \text{ МБ}$$

Так как срок хранения архивов на резервном узле составляет 4 недели, одновременно будут храниться последние 28 ежедневных копий. Через месяц работы системы на резервном узле останутся копии за дни с 3-го по 30-й. Тогда суммарный объем без учета сжатия равен:

$$V = \sum(S_0 + 650 * n), \text{ где } n = 3..30$$

$$V = 28 * S_0 + 650 * (3 + 4 + \dots + 30)$$

$$3 + 4 + \dots + 30 = 462$$

$$V = 28 * S_0 + 650 * 462$$

$$V = 28 * S_0 + 300300 \text{ МБ}$$

Если не учитывать начальный размер базы, прирост суммарного объема хранимых полных резервных копий за месяц составит:

$$300300 \text{ МБ} \approx 293,26 \text{ ГБ}$$

Реальный объем на резервном узле будет меньше из-за gzip-сжатия. Его можно оценить по фактическому коэффициенту сжатия первого архива. В тестовом запуске размер архива составил 2,4 КБ, так как база на момент проверки была небольшой.

Средний объем измененных данных за сутки 750 МБ не прибавляется как отдельный ежедневный инкремент, потому что в данном этапе используется не инкрементальное резервное копирование, а полный SQL Dump. Измененные строки входят в очередную полную копию уже в новом состоянии. Если бы использовалось архивирование WAL или инкрементальные резервные копии, измененные данные существенно влияли бы на объем хранимых журналов или инкрементов.

1.8. Анализ результатов этапа

В результате выполнения этапа была настроена процедура ежедневного полного логического резервного копирования кластера PostgreSQL с основного узла pg111 на резервный узел pg121. Резервная копия создается с помощью pg_dumpall, сжимается через gzip и сразу передается на резервный узел по SSH. На основной системе архив не сохраняется, что соответствует требованиям задания.

На резервном узле настроено хранение архивов в каталоге `/var/db/postgres4/pg_backups/sql_dump`. Устаревшие архивы старше 28 дней автоматически удаляются при каждом запуске сценария. Тестовый запуск подтвердил создание архива, его наличие на резервном узле, отсутствие архива на основном узле, целостность gzip-файла и наличие внутри SQL Dump-копии кластера.

Главный недостаток выбранного способа - рост суммарного объема хранения при ежедневных полных копиях. Даже без учета начального размера базы за месяц работы при ежедневном приросте 650 МБ новых данных потребуется около 293,26 ГБ до сжатия. Однако такой подход прост в реализации и удобен для восстановления, так как каждая резервная копия является самодостаточным SQL-скриптом восстановления состояния кластера на момент создания копии.

Этап 2. Потеря основного узла

На данном этапе был рассмотрен сценарий полной недоступности основного узла `pg111`. Восстановление работы СУБД выполнялось на резервном узле `pg121` с использованием SQL Dump-архива, полученного на первом этапе лабораторной работы.

Так как резервная копия была создана с помощью `pg_dumpall`, архив содержал SQL-скрипт восстановления всего кластера, включая роли, базы данных, табличные пространства и объекты пользовательской базы `fastorangecity`. Перед восстановлением на резервном узле был подготовлен новый каталог данных PostgreSQL, отдельный каталог WAL и каталог для пользовательского табличного пространства `idxspace`.

2.1. Подготовка резервного узла и выбор резервной копии

На резервном узле были заданы переменные окружения для нового восстановленного экземпляра PostgreSQL:

```
[postgres4@pg121 ~]$ export PGDATA="$HOME/ymy46_restored"
[postgres4@pg121 ~]$ export PGPORT="9530"
[postgres4@pg121 ~]$ export PGDATABASE="fastorangecity"
[postgres4@pg121 ~]$ BACKUP_DIR="$HOME/pg_backups/sql_dump"
[postgres4@pg121 ~]$ TABLESPACE_DIR="$HOME/mgg73"
```

Был просмотрен каталог с резервными копиями, созданными на первом этапе:

```
[postgres4@pg121 ~]$ ls -lh "$BACKUP_DIR"
total 14
-rw-r--r--  1 postgres4 postgres  2,4K 26 апр.  11:22 pg111_full_cluster_2026-04-26_11-22-51.sql.gz
-rw-r--r--  1 postgres4 postgres  2,4K 26 апр.  11:26 pg111_full_cluster_2026-04-26_11-26-12.sql.gz
-rw-r--r--  1 postgres4 postgres  2,4K 27 апр.   03:00 pg111_full_cluster_2026-04-27_03-00-00.sql.gz
```

Для восстановления был выбран последний по времени архив:

```
[postgres4@pg121 ~]$ LATEST_BACKUP="$(ls -t
"$BACKUP_DIR"/pg111_full_cluster_*.sql.gz | head -n 1)"
[postgres4@pg121 ~]$ echo "$LATEST_BACKUP"
```

```
/var/db/postgres4/pg_backups/sql_dump/pg111_full_cluster_2026-04-27_03-00-00.sql.gz
```

Целостность архива была проверена с помощью gzip:

```
[postgres4@pg121 ~]$ gzip -t "$LATEST_BACKUP" && echo "backup archive is valid"
backup archive is valid
```

Также было проверено начало SQL Dump-архива:

```
[postgres4@pg121 ~]$ gzip -dc "$LATEST_BACKUP" | head -n 20
--
-- PostgreSQL database cluster dump
--

SET default_transaction_read_only = off;

SET client_encoding = 'WIN1251';
SET standard_conforming_strings = on;

--
-- Roles
--

CREATE ROLE default_user;
ALTER ROLE default_user WITH NOSUPERUSER INHERIT NOCREATEROLE NOCREATEDB LOGIN
NOREPLICATION NOBYPASSRLS PASSWORD 'SCRAM-SHA-256$...';
CREATE ROLE postgres3;
ALTER ROLE postgres3 WITH SUPERUSER INHERIT CREATEROLE CREATEDB LOGIN REPLICATION
BYPASSRLS;
```

Из вывода видно, что архив является дампом кластера PostgreSQL, использует кодировку WIN1251 и содержит восстановление ролей default_user и postgres3.

Далее были подготовлены каталоги для нового кластера и пользовательского табличного пространства:

```
[postgres4@pg121 ~]$ mkdir -p "$PGDATA"
[postgres4@pg121 ~]$ mkdir -p "$TABLESPACE_DIR"
[postgres4@pg121 ~]$ chmod 700 "$PGDATA" "$TABLESPACE_DIR"
[postgres4@pg121 ~]$ ls -ld "$PGDATA" "$TABLESPACE_DIR"
drwx----- 2 postgres4 postgres 2 1 мая 12:55 /var/db/postgres4/mgg73
drwx----- 2 postgres4 postgres 2 1 мая 12:55 /var/db/postgres4/ymy46_restored
```

2.2. Инициализация восстановленного кластера

Для соответствия конфигурации основного узла был подготовлен отдельный каталог WAL. В предыдущей лабораторной работе основной кластер использовал отдельную директорию WAL, поэтому при восстановлении на резервном узле была сохранена аналогичная структура каталогов.

```
[postgres4@pg121 ~]$ export PGWAL="$HOME/xlg69_restored"
[postgres4@pg121 ~]$ mkdir -p "$PGWAL"
[postgres4@pg121 ~]$ chmod 700 "$PGWAL"
```

Новый кластер был инициализирован с той же локалью и кодировкой, что и исходный кластер:

```
[postgres4@pg121 ~]$ rmdir "$PGDATA" 2>/dev/null || true

[postgres4@pg121 ~]$ initdb -D "$PGDATA" -X "$PGWAL" --locale=ru_RU.CP1251 -
-encoding=WIN1251
Файлы, относящиеся к этой СУБД, будут принадлежать пользователю "postgres4".
От его имени также будет запускаться процесс сервера.

Кластер баз данных будет инициализирован с локалью "ru_RU.CP1251".
Выбрана конфигурация текстового поиска по умолчанию "russian".

создание каталога /var/db/postgres4/ymy46_restored... ок
исправление прав для существующего каталога /var/db/postgres4/xlg69_restored... ок
создание подкаталогов... ок
создание конфигурационных файлов... ок
сохранение данных на диске... ок

Готово. Теперь вы можете запустить сервер баз данных:

pg_ctl -D /var/db/postgres4/ymy46_restored -l файл_журнала start
```

Таким образом, на резервном узле был создан новый пустой кластер PostgreSQL с локалью ru_RU.CP1251 и кодировкой WIN1251, совместимый с дампом основного кластера.

2.3. Конфигурация и запуск сервера на резервном узле

После инициализации в файл postgresql.conf были добавлены параметры, аналогичные параметрам основного узла из предыдущей лабораторной работы:

```
[postgres4@pg121 ~]$ cat >> "$PGDATA/postgresql.conf" <<'EOF'

# Настройки восстановленного экземпляра для этапа 2
listen_addresses = 'localhost'
port = 9530

max_connections = 30
shared_buffers = 2GB
temp_buffers = 32MB
work_mem = 64MB
checkpoint_timeout = 15min
effective_cache_size = 18GB
fsync = on
commit_delay = 10000

logging_collector = on
log_destination = 'stderr'
log_directory = 'log'
log_filename = 'postgresql-%Y-%m-%d_%H%M%S.log'
log_min_messages = notice
log_connections = on
log_checkpoints = on
EOF
```

Файл pg_hba.conf был настроен для локальных подключений. Подключения через Unix-domain socket выполняются в режиме peer, TCP/IP-подключения разрешены только с localhost, остальные сетевые подключения явно запрещены:

```
[postgres4@pg121 ~]$ cat > "$PGDATA/pg_hba.conf" <<'EOF'
# TYPE DATABASE USER ADDRESS METHOD
```

```
# Unix-domain socket
local all all peer

# TCP/IP only from localhost
host all all 127.0.0.1/32 trust
host all all ::1/128 trust

# Reject everything else
host all all 0.0.0.0/0 reject
host all all ::0/0 reject
EOF
```

После настройки сервер PostgreSQL был запущен на резервном узле:

```
[postgres4@pg121 ~]$ pg_ctl -D "$PGDATA" start
ожидание запуска сервера...2026-05-01 12:58:32.389 MSK [39786] СООБЩЕНИЕ:
передача вывода в протокол процессу сбора протоколов
2026-05-01 12:58:32.389 MSK [39786] ПОДСКАЗКА: В дальнейшем протоколы будут
выводиться в каталог "log".
    готово
сервер запущен

[postgres4@pg121 ~]$ pg_ctl -D "$PGDATA" status
pg_ctl: сервер работает (PID: 39786)
/usr/local/bin/postgres "-D" "/var/db/postgres4/ymy46_restored"

[postgres4@pg121 ~]$ psql -p "$PGPORT" -d postgres -c '\conninfo'
Вы подключены к базе данных "postgres" как пользователь "postgres4" через сокет в
"/tmp", порт "9530".
```

2.4. Восстановление из SQL Dump-архива

Перед восстановлением было проверено, что в дампе присутствует старый путь пользовательского табличного пространства основного узла:

```
[postgres4@pg121 ~]$ gzip -dc "$LATEST_BACKUP" | grep -n "CREATE
TABLESPACE\|/var/db/postgres3/mgg73" | head -n 20
32:CREATE TABLESPACE idxspace OWNER postgres3 LOCATION '/var/db/postgres3/mgg73';
```

Так как восстановление выполнялось на другом узле и под другим системным пользователем, путь табличного пространства был заменён с /var/db/postgres3/mgg73 на /var/db/postgres4/mgg73. Восстановление было выполнено потоковой командой:

```
[postgres4@pg121 ~]$ gzip -dc "$LATEST_BACKUP" | sed
"s|/var/db/postgres3/mgg73|/var/db/postgres4/mgg73|g" | psql -p "$PGPORT" -d
postgres -v ON_ERROR_STOP=1
SET
SET
SET
CREATE ROLE
ALTER ROLE
CREATE ROLE
ALTER ROLE
CREATE TABLESPACE
GRANT
Вы подключены к базе данных "template1" как пользователь "postgres4".
...
CREATE DATABASE
ALTER DATABASE
```

```

Вы подключены к базе данных "fastorangecity" как пользователь "postgres4".
...
CREATE TABLE
ALTER TABLE
CREATE SEQUENCE
ALTER SEQUENCE
COPY 3
COPY 3
COPY 3
COPY 4
CREATE INDEX
CREATE INDEX
CREATE INDEX
CREATE INDEX
Вы подключены к базе данных "postgres" как пользователь "postgres4".

```

2.5. Проверка доступности данных

После восстановления был проверен список баз данных:

```

[postgres4@pg121 ~]$ psql -p "$PGPORT" -d postgres -c '\l'

```

Имя	Владелец	Кодировка	Провайдер локали	Список баз данных LC_COLLATE
fastorangecity	default_user	WIN1251	libc	ru_RU.CP1251
postgres	postgres4	WIN1251	libc	ru_RU.CP1251
template0	postgres4	WIN1251	libc	ru_RU.CP1251
template1	postgres4	WIN1251	libc	ru_RU.CP1251

(4 строки)

Также был проверен список табличных пространств:

```

[postgres4@pg121 ~]$ psql -p "$PGPORT" -d postgres -c '\db'

```

Имя	Владелец	Расположение
idxspace	postgres3	/var/db/postgres4/mgg73
pg_default	postgres4	
pg_global	postgres4	

(3 строки)

Владелец пользовательского табличного пространства `idxspace` после восстановления остался `postgres3`, поскольку это значение было сохранено в SQL Dump-архиве. При этом физическое расположение табличного пространства было скорректировано на путь резервного узла `/var/db/postgres4/mgg73`.

Далее была проверена доступность восстановленной базы данных и наличие пользовательских таблиц:

```

[postgres4@pg121 ~]$ psql -p "$PGPORT" -d fastorangecity -c '\conninfo'
Вы подключены к базе данных "fastorangecity" как пользователь "postgres4" через
сокет в "/tmp", порт "9530".

```

```
[postgres4@pg121 ~]$ psql -p "$PGPORT" -d fastorangecity -c '\dt'
```

Схема	Имя	Тип	Владелец
public	accounts	таблица	default_user
public	branches	таблица	default_user
public	clients	таблица	default_user
public	operations	таблица	default_user

(4 строки)

Количество восстановленных строк совпало с тестовыми данными из предыдущей лабораторной работы:

```
[postgres4@pg121 ~]$ psql -p "$PGPORT" -d fastorangecity -c "
SELECT 'clients' AS table_name, count(*) FROM clients
UNION ALL
SELECT 'branches', count(*) FROM branches
UNION ALL
SELECT 'accounts', count(*) FROM accounts
UNION ALL
SELECT 'operations', count(*) FROM operations;
"
```

table_name	count
clients	3
branches	3
accounts	3
operations	4

(4 строки)

Отдельно была проверена корректность размещения таблиц и индексов по табличным пространствам:

```
[postgres4@pg121 ~]$ psql -p "$PGPORT" -d fastorangecity -c "
SELECT
  c.relname AS object_name,
  CASE c.relkind
    WHEN 'r' THEN 'table'
    WHEN 'i' THEN 'index'
    WHEN 'S' THEN 'sequence'
    ELSE c.relkind::text
  END AS object_type,
  COALESCE(t.spcname, 'pg_default') AS tablespace
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
LEFT JOIN pg_tablespace t ON t.oid = c.reltablespace
WHERE n.nspname = 'public'
  AND c.relname IN (
    'clients', 'branches', 'accounts', 'operations',
    'idx_accounts_client_id', 'idx_accounts_branch_id',
    'idx_operations_account_id', 'idx_operations_operation_time'
  )
ORDER BY object_type, object_name;
"
```

object_name	object_type	tablespace
idx_accounts_branch_id	index	idxspace
idx_accounts_client_id	index	idxspace
idx_operations_account_id	index	idxspace
idx_operations_operation_time	index	idxspace

accounts	table	pg_default
branches	table	pg_default
clients	table	pg_default
operations	table	pg_default

(8 строк)

Для финальной проверки было выполнено подключение к восстановленной базе от имени пользовательской роли default_user, то есть не от имени администратора восстановленного кластера:

```
[postgres4@pg121 ~]$ psql -h localhost -p "$PGPORT" -U default_user -d
fastorangecity -c '\conninfo'
Вы подключены к базе данных "fastorangecity" как пользователь "default_user"
(сервер "localhost": адрес "::1", порт "9530").
```

После подключения от имени default_user были прочитаны данные из таблиц clients и operations:

```
[postgres4@pg121 ~]$ psql -h localhost -p "$PGPORT" -U default_user -d
fastorangecity -c "SELECT * FROM clients;"
 client_id | surname | name  | gender | registered_at
-----+-----+-----+-----+-----
          1 | Иванов | Иван  | MALE   | 2026-03-01 10:00:00
          2 | Петрова | Анна  | FEMALE | 2026-03-02 11:30:00
          3 | Сидоров | Максим | NOT STATED | 2026-03-03 09:15:00
(3 строки)

[postgres4@pg121 ~]$ psql -h localhost -p "$PGPORT" -U default_user -d
fastorangecity -c "SELECT * FROM operations;"
 operation_id | account_id | operation_type | amount | operation_time
-----+-----+-----+-----+-----
            1 |          1 | Cash deposit   | 5000.00 | 2026-03-08 10:00:00
            2 |          1 | Cash withdrawal | 1000.00 | 2026-03-09 13:00:00
            3 |          2 | Credit         | 7000.00 | 2026-03-10 15:30:00
            4 |          3 | Contribution    | 2500.00 | 2026-03-11 09:40:00
(4 строки)
```

2.6. Анализ результатов этапа

В результате выполнения второго этапа была смоделирована ситуация полной недоступности основного узла pg111. Восстановление выполнялось исключительно на резервном узле pg121 с использованием SQL Dump-архива, созданного на первом этапе.

На резервном узле был инициализирован новый кластер PostgreSQL с локалью ru_RU.CP1251 и кодировкой WIN1251. Конфигурация порта, локальных подключений, параметров памяти и журналирования была выполнена аналогично основному узлу из предыдущей лабораторной работы. После запуска сервера резервный экземпляр стал доступен через Unix-domain socket и TCP/IP localhost.

При восстановлении была выполнена замена пути пользовательского табличного пространства idxspace, так как исходный путь /var/db/postgres3/mgg73 относится к основному узлу и недоступен на резервном. После замены объекты, которые в исходном кластере размещались в idxspace, были восстановлены в каталоге /var/db/postgres4/mgg73.

Проверка показала, что база fastorangecity восстановлена, её владельцем является default_user, пользовательские таблицы clients, branches, accounts и operations доступны, а

количество строк соответствует данным из предыдущей лабораторной работы. Вторичные индексы были восстановлены в табличном пространстве `idxspace`, а сами таблицы остались в `pg_default`.

Дополнительная проверка подключения от имени `default_user` подтвердила, что восстановленная база доступна не только администратору кластера, но и пользовательской роли, от имени которой в предыдущей лабораторной работе выполнялось наполнение базы. Таким образом, при полной недоступности основного узла работа СУБД была успешно восстановлена на резервном узле.